



Museum of Broken Relationships

Relatório - Projeto Final

Unidade Curricular: Desenvolvimento de Aplicações Web
Instituição: Instituto Politécnico de Setúbal - ESCE
Ano Letivo: 2025/2026
Grupo: Carolina Fernandes, Mariana Antunes e Pedro Figueiredo

Índice

1. Introdução	3
2. Objetivos do Projeto	4
3. Conceito e Mecânica do Jogo.....	4
3.1 Amor-Próprio.....	4
3.2 Espaços de Cura.....	5
3.3 Momento Bónus.....	5
3.4 Conclusão do Jogo	5
4. Arquitetura da Aplicação.....	5
4.1 Estrutura de Ficheiros.....	6
5. Backend	7
5.1 Tecnologias	7
5.2 Base de Dados	7
5.3 Autenticação e Passwords	7
5.4 Validação das Regras do Jogo.....	8
6. Frontend.....	8
6.1 Templates e Interface.....	8
6.2 Comunicação com o Backend	9
6.3 Temporizadores	9
6.4 Tema Claro e Escuro	9
6.5 Rodapé e Controlo do Scroll.....	9
7. Rotas da Aplicação.....	10
8. Estados dos Slots.....	10
9. Funcionalidades Implementadas.....	11
10. Segurança.....	12
11. Como Executar.....	12
12. Resultados e Aprendizagens	13
14. Conclusão	13

1. Introdução

O Museum of Broken Relationships é um jogo web desenvolvido no âmbito da unidade curricular de Desenvolvimento de Aplicações Web. O projeto utiliza Flask e SQLite no backend e HTML, CSS, JavaScript e templates Jinja no frontend.

O tema era livre, mas os requisitos técnicos eram fixos: autenticação de utilizadores, construções com slots, recursos que evoluem ao longo do tempo, tarefas com temporizadores e interação com outros jogadores. Decidimos ir um pouco além do que era pedido e criar algo com uma história por trás.

O nome é inspirado num museu real em Zagreb, na Croácia, onde as pessoas doam objetos de relações terminadas e as histórias que ficaram para trás.

Escolhemos criar o Museum of Broken Relationships um jogo sobre recuperar de uma relação que terminou. Em vez de gerir ouro ou madeira, o utilizador gere o seu próprio amor-próprio e lágrimas através de decisões emocionais.

Decidimos apostar na criatividade e numa ideia com personalidade, pois a maioria dos jogos de gestão que conhecemos são sobre acumular recursos mais ouro, mais madeira, mais população. Queríamos inverter essa lógica: e se o recurso fosse algo intangível, como o bem-estar emocional? E se as decisões do jogo refletissem escolhas reais que as pessoas fazem quando estão a tentar superar uma separação?

O jogo adapta mecânicas tradicionais de gestão e progressão a um tema emocional. O utilizador gere dois recursos: Amor-Próprio, representado por uma pontuação entre 0 e 100, e Lágrimas, utilizadas para preparar espaços e tomar decisões. Cada utilizador começa com 4 pontos de Amor-Próprio e 60 Lágrimas.

O Museum of Broken Relationships é um projeto de que nos orgulhamos. Conseguimos criar algo funcional, com uma identidade visual coerente e uma mecânica de jogo que faz sentido de ponta a ponta.

2. Objetivos do Projeto

O objetivo principal foi desenvolver uma aplicação web interativa que permitisse aplicar os conteúdos estudados na unidade curricular:

- ♥ Autenticação e sessões de utilizador;
- ♥ Persistência de dados numa base de dados relacional;
- ♥ Comunicação assíncrona entre frontend e backend;
- ♥ Manipulação dinâmica do DOM;
- ♥ Validação de ações no servidor;
- ♥ Temporizadores e progressão de tarefas;
- ♥ Interação indireta entre jogadores através de uma classificação;
- ♥ Design responsivo e temas visuais.

3. Conceito e Mecânica do Jogo

3.1 Amor-Próprio

O Amor-Próprio é o recurso central do jogo. A pontuação é limitada pelo backend ao intervalo entre 0 e 100.

As decisões consideradas positivas aumentam normalmente a pontuação em 8 pontos. As decisões negativas reduzem normalmente 5 pontos, existindo uma decisão bónus que pode reduzir 10 pontos.

O estado emocional apresentado ao utilizador varia de acordo com a pontuação:

Pontuação	Estado emocional
0 a 20	Desolado
21 a 40	A Sofrer
41 a 60	A Recuperar
61 a 80	Confiante
81 a 99	Quase Curado
100	Curado

Quando o Amor-Próprio ou as Lágrimas chegam a zero, o utilizador pode reiniciar o progresso. O reinício repõe os recursos iniciais e coloca os quatro espaços no estado vazio.

3.2 Espaços de Cura

Cada utilizador possui quatro slots fixos, associados aos seguintes espaços:

- ♥ **Baú das Recordações Físicas:** roupas, cartas e presentes;
- ♥ **Arquivo Digital:** fotografias, mensagens e redes sociais;
- ♥ **A Mente e os Pensamentos:** pensamentos repetidos, autocuidado e apoio dos amigos;
- ♥ **Novos Começos:** hobbies, exercício físico e novas experiências.

Cada espaço possui três momentos principais apresentados sequencialmente. Uma resposta positiva permite avançar para o momento seguinte depois da recolha. Uma resposta negativa reduz o Amor-Próprio e obriga o utilizador a repetir o mesmo momento.

Antes de utilizar um espaço, o jogador paga um custo em Lágrimas e aguarda o respetivo tempo de preparação. Cada decisão custa uma Lágrima. Ao recolher uma resposta correta, o jogador recupera duas Lágrimas.

3.3 Momento Bónus

Depois de concluir o momento “Redes sociais”, é desbloqueado um momento adicional chamado “Novas publicações”. Este momento é adicionado ao Arquivo Digital e adapta a situação e as opções apresentadas de acordo com a decisão tomada anteriormente.

O desbloqueio e a decisão anterior são guardados na base de dados, permitindo manter a progressão depois do logout ou de uma nova visita.

3.4 Conclusão do Jogo

Um espaço é finalizado quando todos os seus momentos são respondidos corretamente e recolhidos. Quando os quatro espaços ficam finalizados, o backend define o Amor-Próprio como 100 e o frontend apresenta um popup especial de conclusão.

4. Arquitetura da Aplicação

A aplicação segue o padrão MVC através de uma separação física e lógica:

- ♥ **Model:** models/game_model.py contém o acesso à base de dados, validações, dados e regras do jogo;
- ♥ **View:** templates/ e static/ contém a interface apresentada no browser;
- ♥ **Controller:** controllers/main_controller.py recebe os pedidos HTTP, coordena o Model e devolve páginas ou respostas JSON.

O ficheiro app.py funciona apenas como ponto de entrada: cria e configura a aplicação Flask, regista o Controller e inicializa a base de dados.

4.1 Estrutura de Ficheiros

```
museum_broken_relationships/
|-- app.py
|-- requirements.txt
|-- README.md
|-- controllers/
|   |-- __init__.py
|   `-- main_controller.py
|-- models/
|   |-- __init__.py
|   `-- game_model.py
|-- templates/
|   |-- base.html
|   |-- index.html
|   |-- login.html
|   |-- register.html
|   `-- dashboard.html
`-- static/
    |-- css/
    |   `-- style.css
    |-- img/
    |   |-- arquivodigital.png
    |   |-- bau.png
    |   |-- branch_landing.gif
    |   |-- fundo.jpeg
    |   |-- fundo_escuro.jpeg
    |   |-- mentepensamentos.png
    |   |-- novoscomecos.png
    |   `-- pixel_heart.png
    `-- js/
        |-- jogo.js
        `-- tema.js
```

5. Backend

5.1 Tecnologias

O backend utiliza:

- ♥ Python;
- ♥ Flask 3.1.0;
- ♥ SQLite;
- ♥ Flask-Login 0.6.3;
- ♥ Passlib 1.7.4.

5.2 Base de Dados

A base de dados é criada automaticamente quando a aplicação é iniciada. Existem duas tabelas principais.

Tabela	Campos principais	Finalidade
user	id, username, email, password, publicacoes_disponivel, publicacoes_resolvido, silenciou_ex	Guarda utilizadores, credenciais, recursos e progressão condicional
slot	id, user_id, numero, estado, tipo, etapa, construcao_fim, tarefa_fim, tarefa_nome, tarefa_correta	Guarda a preparação e progressão dos espaços

O código inclui ainda uma atualização simples do esquema para adicionar os campos etapa e tarefa_correta a bases de dados antigas.

5.3 Autenticação e Passwords

O registo valida se o username e o email estão preenchidos, se o email contém o símbolo @, se a password possui pelo menos seis caracteres e se o username ou email já existem.

As novas passwords são protegidas com pbkdf2_sha256, através da biblioteca Passlib. O sistema também reconhece hashes antigos em formato SHA-256 hexadecimal. Após um login válido com um hash antigo, a password é automaticamente convertida para pbkdf2_sha256.

A autenticação utiliza Flask-Login sobre sessões Flask. Depois do login ou registo, o utilizador permanece autenticado entre pedidos. O logout termina a autenticação e limpa a sessão.

5.4 Validação das Regras do Jogo

O backend é a fonte oficial do estado do jogo. Antes de alterar dados, as rotas verificam:

- ♥ Se o utilizador está autenticado;
- ♥ Se o slot pertence ao utilizador;
- ♥ Se o slot se encontra no estado esperado;
- ♥ Se o tipo, tarefa e opção recebidos são válidos;
- ♥ Se a tarefa corresponde à etapa atual;
- ♥ Se o Amor-Próprio permite continuar;
- ♥ Se o temporizador já terminou antes da recolha.

Os dados das tarefas são enviados para o frontend para apresentação, mas o impacto final e a atualização da base de dados são sempre calculados e validados novamente no servidor.

6. Frontend

6.1 Templates e Interface

O frontend utiliza templates Jinja. O ficheiro base.html contém a estrutura comum, a ligação ao CSS, o conteúdo principal, o rodapé e os scripts partilhados.

O dashboard apresenta:

- ♥ Estado emocional e barra de Amor-Próprio;
- ♥ Quatro espaços de cura;
- ♥ Classificação dos dez utilizadores com maior pontuação;
- ♥ Botão de logout;
- ♥ Alternância entre tema claro e escuro;
- ♥ Modais de decisão;
- ♥ Notificações de resultado;
- ♥ Popup de conclusão.

6.2 Comunicação com o Backend

O ficheiro `static/js/jogo.js` utiliza a Fetch API para comunicar com as rotas da API sem submeter formulários tradicionais. Depois de cada resposta, o JavaScript atualiza a barra de progresso, o valor do Amor-Próprio, o estado emocional e as notificações.

O frontend também consulta `/api/estado` a cada 15 segundos. Este polling permite atualizar os dois recursos e detetar alterações no estado dos slots.

6.3 Temporizadores

Os espaços possuem temporizador de preparação e cada tarefa possui atualmente uma duração curta adequada à demonstração. Quando uma decisão é submetida:

1. O backend calcula imediatamente o impacto no Amor-Próprio;
2. O backend guarda a data e hora de conclusão em ``tarefa_fim``;
3. O slot passa para o estado ``processando``;
4. O frontend consulta ``/api/verificar_tarefa`` e apresenta uma contagem decrescente;
5. Quando o tempo termina, o slot passa para ``concluida``;
6. O utilizador recolhe o resultado para avançar ou repetir o momento.

Como a data final é guardada e validada pelo servidor, atualizar a página ou alterar o contador no browser não permite recolher antecipadamente.

6.4 Tema Claro e Escuro

O ficheiro `static/js/tema.js` permite alternar entre tema claro e escuro. A preferência é guardada no `localStorage` com a chave `preferred-theme`, permanecendo ativa entre visitas no mesmo browser.

6.5 Rodapé e Controlo do Scroll

O rodapé utiliza posicionamento fixo para permanecer visível sobre a imagem de fundo. No dashboard, a página utiliza `100dvh`, `overflow: hidden` e `overscroll-behavior: none` para ocupar a área visível e impedir o arrastamento para uma área branca abaixo do fundo.

Em desktop, o dashboard permanece limitado ao ecrã visível. Em tablet e mobile, o layout passa para uma coluna e permite scroll vertical para garantir acesso a todo o conteúdo.

7. Rotas da Aplicação

Rota	Método	Função
/	GET	Apresenta a página inicial ou redireciona utilizadores autenticados
/register	GET, POST	Apresenta e processa o registo
/login	GET, POST	Apresenta e processa o login
/logout	GET	Limpa a sessão e termina a autenticação
/dashboard	GET	Carrega o estado do jogo e a classificação
/api/construir	POST	Inicia a preparação temporizada de um espaço vazio
/api/dar_ordem	POST	Valida e processa uma decisão
/api/recolher	POST	Recolhe o resultado e atualiza a progressão
/api/verificar_tarefa	POST	Verifica o estado do temporizador
/api/estado	GET	Devolve a pontuação e o estado dos slots
/reiniciar	POST	Reinicia o jogo quando o Amor-Próprio ou as Lágrimas chegam a zero

8. Estados dos Slots

Estado	Significado
Vazio	O espaço ainda não foi iniciado
Construindo	O espaço está a ser preparado e ainda não permite decisões
Ativo	Existe um momento disponível para responder
Processando	A decisão foi submetida e o temporizador está ativo
Concluída	O temporizador terminou e o resultado pode ser recolhido
Finalizado	Todos os momentos do espaço foram concluídos corretamente

9. Funcionalidades Implementadas

Funcionalidade	Estado
Registo, login, logout e sessões	Implementado
Hashing de passwords com migração de hashes antigos	Implementado
Autenticação e proteção de rotas com Flask-Login	Implementado
Dois recursos: Amor-Próprio e Lágrimas	Implementado
Custos e tempo de preparação dos espaços	Implementado
Quatro espaços de cura com progressão sequencial	Implementado
Amor-Próprio limitado entre 0 e 100	Implementado
Estados emocionais baseados na pontuação	Implementado
Temporizadores validados no servidor	Implementado
Recolha e repetição de momentos	Implementado
Momento bónus adaptado à decisão anterior	Implementado
Leaderboard com os dez melhores utilizadores	Implementado
Tema claro e escuro persistente	Implementado
Reinício quando o Amor-Próprio ou as Lágrimas chegam a zero	Implementado
Popup após conclusão dos quatro espaços	Implementado
Responsividade em desktop, tablet e mobile	Implementado
Testes automatizados dos fluxos principais	Implementado

10. Segurança

O projeto inclui várias medidas adequadas ao contexto académico:

- ♥ Passwords protegidas com hashing;
- ♥ Queries SQL parametrizadas;
- ♥ Validação das ações do jogo no backend;
- ♥ Verificação de autenticação nas rotas protegidas;
- ♥ Limitação da pontuação entre 0 e 100.

Existem, no entanto, limitações que devem ser consideradas numa versão de produção:

- ♥ Existe uma chave secreta de desenvolvimento como valor de fallback quando a variável de ambiente não é definida;
- ♥ Não existe proteção CSRF nos formulários e pedidos da API;
- ♥ Não existe limitação de tentativas de login;
- ♥ Os tempos curtos utilizados facilitam a demonstração, mas devem ser configuráveis numa versão de produção.

11. Como Executar

Para executar a aplicação localmente, é necessário garantir que o Python se encontra instalado no computador. O primeiro passo consiste em abrir um terminal e aceder à pasta onde se encontra o projeto:

```
cd museum_broken_relationships
```

De seguida, deve ser criado um ambiente virtual. Este ambiente permite isolar as dependências do projeto, evitando conflitos com outras aplicações Python instaladas no sistema.

```
python -m venv venv
```

Após a criação do ambiente virtual, este deve ser ativado.

No Windows: `venv\scripts\activate`

No macOS: `source venv/bin/activate`

Com o ambiente virtual ativo, é necessário instalar todas as bibliotecas utilizadas pelo projeto. Estas dependências encontram-se definidas no ficheiro requirements.txt.

```
pip install -r requirements.txt
```

Depois de concluída a instalação das dependências, a aplicação pode ser iniciada através do seguinte comando:

```
python app.py
```

12. Resultados e Aprendizagens

O projeto permitiu aplicar os principais conceitos de desenvolvimento web estudados na unidade curricular: autenticação, sessões, persistência, templates, manipulação do DOM, pedidos assíncronos e validação no servidor.

A implementação demonstrou a importância de manter o backend como fonte oficial do estado, sobretudo em funcionalidades com pontuação, progressão e temporizadores. Também evidenciou a necessidade de equilibrar apresentação visual, experiência do utilizador, responsividade e segurança.

14. Conclusão

O Museum of Broken Relationships apresenta uma interpretação original de um jogo de gestão e progressão. A aplicação permite criar uma conta, autenticar, tomar decisões, acompanhar a evolução do Amor-Próprio, completar quatro espaços e comparar resultados através de uma classificação.

O projeto cumpre os principais objetivos funcionais definidos e inclui funcionalidades adicionais, como o tema escuro, a migração de hashes antigos, o momento bónus condicionado por uma decisão anterior e o popup de conclusão.

Apesar de existirem limitações a aplicação constitui uma base funcional e coerente.

O tema escolhido foi um risco que valeu a pena. Deu-nos motivação extra para cuidar dos detalhes, desde o texto das mensagens de feedback até à paleta de cores e aos ícones pixel art de cada espaço.